

In-class assignment - Async Basics Demo (Timeout, Promise, Await, Fetch)

Part A: Setup and First Run

Timeout Output:

When clicking Async Timeout, the output is

```
Start
End
Timeout finished after 2 seconds
```

Explanations:

The function is scheduled so the program continues immediately.

Status Line for Promise / Await / Fetch

- Promise button gives Status: Waiting (Promise)... then back to Status: Idle
- Await button gives Status: Waiting (await)... then back to Status: Idle
- Fetch button gives Status: Loading from API... then back to Status: Idle

PART B — Async Timeout

We change the delay from 2000 ms to 500 ms.

```
setTimeout(function () {
  log("Timeout finished after 2 seconds");
}, 500);
```

Output:

"End" will still appear before "Timeout finished" because we are waiting for setTimeout, the program will continue executing the rest of the code immediately.

PART C — Promise

Task C1 — Change resolve message:

```
resolve("Promise resolved after 1 second!");
```

This message appears correctly after waiting.

Task C2 — Change delay to 2000 ms:

```
setTimeout(function () {  
  resolve("Promise resolved after 1 second!");  
}, 2000);
```

While waiting the output, i tried to click on other buttons, they reacted immediately before the resolution. So the page remained responsive.

".then()" runs after the promise is resolved and is executed when the promise finishes successfully and returns a value.

PART D — Async/Await

Task D1 — Change status text:

```
setStatus("Please wait, async/await running...");
```

Changing the text does not bother the program (it works correctly).

Task D2 — Add logs before and after await

```
async function runAwaitExample() {  
  
  clearOutput();  
  setStatus("Please wait, async/await running...");  
  
  log("Before await");  
  
  const result = await waitOneSecond();  
  
  log("After await");  
  log(result);  
  
  setStatus("Idle");  
}
```

Output - orders of logs:

```
Before await  
After await  
Promise resolved after 1 second!
```

What does await do?

`await` pauses the execution of the asynchronous function until the Promise resolves, without blocking the browser.

PART E — Fetch

Task E1 — Change output to ID + Title:

After modifying this part this part:

```
log("ID: " + data.id);  
log("Title: " + data.title);
```

Now it prints:

```
ID: 1  
Title: delectus aut autem
```

Task E2 — Force HTTP error:

Change URL to:

```
"https://jsonplaceholder.typicode.com/todos/999999"
```

Output:

This link provides an error 404 (not found) since this link does not exist.

```
Error: HTTP Error: 404
```

Check response.ok :

We check `response.ok` to detect HTTP errors (like 404), because `fetch()` does not automatically throw an error for them.

PART F — Reflection Answers

Why do we use async/await?

We use `async/await` to write asynchronous code in a cleaner and more readable way. It makes Promises easier to understand and avoids long chains of `.then()` calls.

Why do we use try/catch with fetch?

We use `try/catch` to handle network errors and manually thrown errors so that the program does not crash and we can display a meaningful message.

Why do we check `response.ok`?

We check `response.ok` because `fetch` only rejects on network failure, not on HTTP errors. Checking `response.ok` allows us to detect server errors like 404 or 500.